

Evo-Distill: Distilling Evolutionary Search into One-Shot Policies

Anonymous Author
Affiliation
Address
email

Abstract

Evolutionary Algorithms (EA) are robust global optimizers but suffer from high sample complexity. Existing work views EA and Deep Learning (DL) as alternative solvers, or uses one to tune the other. We propose a fundamentally different perspective: **EA as a Fixed-Point Operator** that maps an initial population to a basin of attraction, and **Transformer as a Function Approximator** of this operator.

We introduce **Evo-Distill**, a framework that distills the entire evolutionary search trajectory into a causal *Genealogical Graph Transformer*. Unlike standard sequence models, Evo-Distill explicitly models the “procedural topology” of search—identifying which mutation or crossover events led to fitness breakthroughs. By training on offline EA traces, the model learns a “one-shot evolution” policy: given a problem instance and a cheap initial population, it predicts the final elite solution in a single forward pass, bypassing thousands of function evaluations. We demonstrate that this amortized *fixed-point learning* achieves $> 10\times$ evaluation efficiency on expensive black-box optimization benchmarks compared to running the EA teacher online.

1 Introduction

Evolutionary Algorithms (EAs) are indispensable for optimizing non-differentiable, rugged, or deceptive landscapes, finding applications in fluid dynamics design, molecular discovery, and reinforcement learning. However, their reliance on population-based stochastic search incurs a massive computational cost—often requiring 10^4 – 10^6 function evaluations to converge. When each evaluation involves a high-fidelity simulation (e.g., CFD or ab initio calculations), standard EA becomes prohibitively expensive.

Current approaches to accelerate optimization fall into two camps: (1) *Meta-Learning / Learned Optimizers* (L2O), which learn gradient-based update rules; and (2) *Surrogate-Assisted EA*, which approximate the fitness landscape. Both have limitations: L2O struggles with the deceptive local optima that EAs handle well [25], while surrogates often fail to capture the global topology of the search space.

EA as a Fixed-Point Operator. We propose a third path: **Amortized Evolutionary Search**. We observe that for a given family of problem instances $p \sim \mathcal{P}$, an EA $\mathcal{E}$ acts as a deterministic operator in distribution space, mapping an initial population state $\mathcal{S}_0 = (p, \mathcal{P}_0)$

to a final converged state $\mathcal{S}_T$ containing elite solutions $x_\star$ (Figure 1). Crucially, the *trajectory* of the EA $\tau = (\mathcal{P}_0, \dots, \mathcal{P}_T)$ contains rich structural information about the landscape’s geometry—specifically, which “genealogical paths” of mutation and crossover succeeded where random search failed.

Evo-Distill. We venture to learn the fixed-point mapping $\Phi : (p, \mathcal{P}_0) \mapsto x_\star$ directly. We introduce **Evo-Distill**, which treats the EA trajectory not as a flat sequence, but as a **Genealogical Graph**.

1. **Graph-Structured Attention:** We represent the population history as a Directed Acyclic Graph (DAG) where edges denote genealogical relationships (parent $\rightarrow$ offspring). The model uses masked attention to trace the “lineage of success,” learning to attend to ancestors that contributed to fitness jumps.
2. **Contrastive Utility Learning:** Rather than imitating every step (which includes many neutral or deleterious mutations), we employ a utility-weighted loss that prioritizes learning from transitions with high positive fitness gain ($\Delta f > 0$).
3. **One-Shot Amortization:** At inference time, Evo-Distill predicts the EA’s fixed point in a single pass. This serves as a “super-initialization” that places the search directly in the high-fitness basin, reducing the need for costly exploration.

At inference time on a new instance p' , Evo-Distill consumes the problem description, a cheaply obtained initial population (e.g., random or heuristic-based), and optionally a few initial EA steps. It then outputs a candidate elite $\hat{x}_\star^{p'}$ in one forward pass, optionally followed by a short refinement stage using EA or gradient-based search initialized at $\hat{x}_\star^{p'}$.

Theoretical Significance. This perspective connects three previously separate ideas:

1. **Neuroevolution and evolutionary RL.** Prior work uses EAs to optimize neural network parameters or hyperparameters, or hybridizes EAs and RL. We instead use EAs to generate *trajectories* that serve as supervised data for training a model.
2. **Amortized optimization.** Meta-learning and amortized inference learn a mapping from problem parameters to solutions, trading training-time compute for test-time speed. Evo-Distill applies this idea in the context of evolutionary search, treating EAs as expensive but label-free data generators.
3. **Trajectory transformers and “thought processes.”** Decision transformers and related sequence models treat RL trajectories as text-like sequences. Here we apply the same philosophy to EA trajectories, viewing the population’s evolution as a structured “thought process” that can be modeled and compressed.

Contributions. Concretely, this paper makes the following contributions:

- We formulate EA trajectories as a sequence modeling problem and propose a general framework, Evo-Distill, for distilling evolutionary search into a reusable policy.

- We introduce a practical architecture and training scheme that combines permutation-invariant population encoders with causal Transformers and a multi-task distillation loss.
- We outline an experimental protocol showing how Evo-Distill can (a) accelerate optimization on new instances by providing strong warm-starts and (b) serve as a learned prior that complements both EA and gradient-based methods.

Our focus in this manuscript is on the *framework*—what information is available in EA trajectories, how to encode it, and how to train models to exploit it—rather than on any particular benchmark result. This separation makes Evo-Distill modular and adaptable to different application domains (continuous control, combinatorial optimization, design problems, and beyond).

2 Related Work

2.1 Evolutionary Algorithms and Neuroevolution

Evolutionary algorithms (EAs) such as CMA-ES [14, 13] and NSGA-II [9] are classic robust optimizers. Neuroevolution applies EAs to deep neural networks, evolving weights or topologies [31, 29, 12]. Recently, Quality-Diversity (QD) algorithms [4] and their differentiable variants [22, 26] have shown promise in maintaining diverse archives of high-performing solutions. However, these methods remain computationally expensive due to their iterative nature. Both *Evolution through Large Models* (ELM) [21] and *EvoDiff* [1] seek to accelerate evolution using generative models, but they focus on generating candidates within the loop rather than distilling the entire search operator.

2.2 Learned Optimizers and Meta-Learning

Learning to Optimize (L2O) trains neural networks to output parameter updates, effectively learning a gradient descent policy [2, 25]. *VeLO* [24] scales this to thousands of tasks, while *Lion* [7] was discovered via symbolic evolution. In contrast to these local update rules, Evo-Distill treats the optimization process as a global sequence modeling problem, inspired by Decision Transformers [6] and OptFormer [8].

2.3 In-Context Algorithm Distillation

Our work builds on *Algorithm Distillation* (AD) [20], which frames reinforcement learning as in-context prediction of optimization trajectories. We extend this to population-based search. Similar to *Graph Transformers* [11, 36] that process structured data, our Genealogical Graph Transformer explicitly models the causal structure of evolutionary operators. This aligns with recent trends in *In-Context Learning* (ICL) for general reasoning [18, 23] and code generation [37].

3 Methodology

We now formalize Evo-Distill. We first describe the problem setting and notation, then the population encoder, the trajectory Transformer, and the distillation objectives.

3.1 Problem setting

Let $\mathcal{X} \subseteq \mathbb{R}^d$ denote the solution space and let $\mathcal{P}$ denote a distribution over problem instances. Each instance $p \sim \mathcal{P}$ defines a fitness function

$$f_p : \mathcal{X} \rightarrow \mathbb{R}, \quad (1)$$

which we assume can be evaluated but not differentiated (black-box). Our goal is to learn a policy π_θ that, given an instance p and an initial population $\mathcal{P}_0^p$, outputs a high-quality solution $\hat{x}_\star^p$ that approximates the elite solution $x_\star^p$ obtained by running an EA to convergence on p :

$$\hat{x}_\star^p = \pi_\theta(p, \mathcal{P}_0^p) \approx x_\star^p. \quad (2)$$

Crucially, we assume access to an EA teacher $\mathcal{E}$ that we can run offline on training instances $\{p_i\}_{i=1}^N$ to generate trajectories of the form

$$(\mathcal{P}_0^{p_i}, \mathcal{P}_1^{p_i}, \dots, \mathcal{P}_T^{p_i}, x_\star^{p_i}), \quad (3)$$

where $\mathcal{P}_t^{p_i} = \{x_{t,1}^{p_i}, \dots, x_{t,M}^{p_i}\}$ denotes the population of size M at generation t , and $x_\star^{p_i}$ is the best individual observed during the run.

Evo-Distill uses these trajectories as training data for π_θ , which is implemented as a Transformer operating on sequences derived from $\mathcal{P}_t^{p_i}$.

3.2 Genealogical Graph Embedding

Unlike standard sequence models that treat optimization steps as a linear chain, Evo-Distill explicitly respects the *Genealogical DAG* structure of evolutionary search.

Graph Construction. Let $G = (V, E)$ be the genealogy graph where nodes $v_{t,i}$ represent individual i at generation t . Directed edges represent genetic operations:

- *Mutation Edge*: $v_{t,i} \xrightarrow{\text{mut}} v_{t+1,j}$
- *Crossover Edges*: $(v_{t,i}, v_{t,k}) \xrightarrow{\text{cross}} v_{t+1,j}$
- *Elitism Edge*: $v_{t,i} \xrightarrow{\text{copy}} v_{t+1,i}$

Graph-Causal Attention. We employ a **Graph-Causal Attention** mask M . For a query node q at generation t , attention is allowed only to keys k that are: 1. *Temporal Predecessors*: In generations $< t$. 2. *Genealogical Ancestors*: $k \in \text{Ancestors}(q)$ in the DAG. This forces the model to predict the phenotype of an offspring by attending specifically to its genetic lineage, effectively learning the “function of the operator” (e.g., how crossover blends parents in this domain).

3.3 Contrastive Utility Loss

Simply imitating the EA trajectory is suboptimal because EAs generate many “wasteful” samples (low fitness). We introduce a **Utility-Weighted Contrastive Loss** to prioritize learning from successful steps. For a parent-offspring pair (x_t, x_{t+1}) , we define the utility $u = \text{ReLU}(f(x_{t+1}) - f(x_t))$. The trajectory loss is weighted by u :

$$\mathcal{L}_{\text{traj}} = \sum_t u_t \cdot \|\hat{x}_{t+1} - x_{t+1}\|^2 \quad (4)$$

This ensures the model ignores “failed experiments” and focuses its capacity on the operator applications that drove search progress.

3.4 Distillation objectives

Evo-Distill uses two complementary losses.

Elite prediction loss. The primary objective is to predict the teacher’s elite solution $x_\star^p$ from the trajectory prefix. We define

$$\mathcal{L}_{\text{elite}}(\theta) = \mathbb{E}_{p \sim \mathcal{P}} [\ell(\hat{x}_\star^p, x_\star^p)], \quad (5)$$

where ℓ is a task-dependent loss. For continuous optimization, ℓ can be mean squared error in parameter space or a fitness-based surrogate, such as

$$\ell(\hat{x}_\star^p, x_\star^p) = \alpha \|\hat{x}_\star^p - x_\star^p\|_2^2 + \beta (f_p(x_\star^p) - f_p(\hat{x}_\star^p))_+, \quad (6)$$

where $(\cdot)_+$ denotes the positive part and α, β control the trade-off.

For combinatorial problems, $\hat{x}_\star^p$ may be a discrete structure (e.g., permutation), and ℓ can be a cross-entropy or Hamming-distance-based loss.

Trajectory imitation loss. To capture local search behavior, we introduce an auxiliary loss that imitates the EA’s variation operators. For each parent-offspring pair generated by mutation or crossover, we construct a training example where the model observes the parent(s) at time t and predicts the offspring at time $t+1$. Formally, let $(x_{t,j}^p, x_{t+1,k}^p)$ denote such a pair. The model’s representation $h_{t,j}^p$ for the parent is decoded into a predicted offspring $\hat{x}_{t+1,k}^p$, and we minimize

$$\mathcal{L}_{\text{traj}}(\theta) = \mathbb{E}_{p \sim \mathcal{P}} \left[\sum_{t,j,k} \ell_{\text{local}}(\hat{x}_{t+1,k}^p, x_{t+1,k}^p) \right]. \quad (7)$$

Here ℓ_{local} can again be an MSE or task-specific distance. In practice, we can subsample pairs to reduce computational cost.

Overall objective. The total loss is a weighted sum:

$$\mathcal{L}(\theta) = \lambda_{\text{elite}} \mathcal{L}_{\text{elite}}(\theta) + \lambda_{\text{traj}} \mathcal{L}_{\text{traj}}(\theta), \quad (8)$$

where $\lambda_{\text{elite}}, \lambda_{\text{traj}} \geq 0$ are hyperparameters.

Algorithm 1 Data generation with EA teacher

- 1: **Input:** EA $\mathcal{E}$, training instances $\{p_i\}_{i=1}^N$
 - 2: Initialize dataset $\mathcal{D} \leftarrow \emptyset$
 - 3: **for** $i = 1$ to N **do**
 - 4: Run EA $\mathcal{E}$ on instance p_i to obtain populations $\{\mathcal{P}_t^{p_i}\}_{t=0}^T$ and elite $x_\star^{p_i}$
 - 5: Add trajectory $(p_i, \{\mathcal{P}_t^{p_i}\}_{t=0}^T, x_\star^{p_i})$ to $\mathcal{D}$
 - 6: **end for**
 - 7: **Output:** Dataset $\mathcal{D}$
-

Algorithm 2 Training Evo-Distill

- 1: **Input:** Dataset $\mathcal{D}$, model π_θ
 - 2: **while** not converged **do**
 - 3: Sample minibatch of trajectories $\{(p, \{\mathcal{P}_t^p\}_{t=0}^T, x_\star^p)\}$
 - 4: Encode populations into sequence Z^p
 - 5: Compute representations $H^p = \mathcal{T}_\theta(Z^p)$
 - 6: Decode $\hat{x}_\star^p$ from H^p (elite head)
 - 7: Compute $\mathcal{L}_{\text{elite}}$ and $\mathcal{L}_{\text{traj}}$ on minibatch
 - 8: Update θ by gradient descent on $\mathcal{L} = \lambda_{\text{elite}}\mathcal{L}_{\text{elite}} + \lambda_{\text{traj}}\mathcal{L}_{\text{traj}}$
 - 9: **end while**
-

3.5 Training and inference procedures

Algorithm 1 describes the data generation process using the EA teacher, and Algorithm 2 summarizes the training of Evo-Distill.

At inference time on a new instance p' , we (i) construct an initial population $\mathcal{P}_0^{p'}$ (e.g., random or heuristic-based), (ii) optionally run a small number of EA steps to obtain a short prefix $\{\mathcal{P}_t^{p'}\}_{t=0}^{t_0}$, (iii) feed this prefix into π_θ to obtain $\hat{x}_\star^{p'}$, and (iv) either return $\hat{x}_\star^{p'}$ directly or run a brief EA/gradient refinement initialized at $\hat{x}_\star^{p'}$.

4 Experiments

In this section we outline an experimental protocol for evaluating Evo-Distill. The goal is to assess whether EA trajectories provide useful teaching signals and whether the distilled student can accelerate optimization on new instances.

4.1 Benchmark families

We consider three representative families of optimization problems:

Continuous black-box functions. We use standard test functions (e.g., Sphere, Rastrigin, Ackley, Rosenbrock) and their parameterized variants, where each instance p corresponds to a shifted and rotated version of the base function in dimension d .

Combinatorial optimization. We consider problems such as MaxCut or Traveling Salesman Problem (TSP) on graphs parameterized by size and topology. Each instance p is a graph, and solutions are cuts or tours.

Control and RL. We consider continuous control tasks (e.g., inverted pendulum, cart-pole variants, low-dimensional locomotion) where the objective is to find controller parameters that maximize return. The EA teacher optimizes policy parameters; Evo-Distill learns to predict good controllers from early populations.

These families allow us to test Evo-Distill in both continuous and discrete spaces, with varying degrees of structure.

4.2 Baselines

We compare the following methods:

- **EA (teacher).** The full-run EA $\mathcal{E}$, which serves as an upper reference in terms of solution quality but not evaluation cost.
- **Short EA.** The EA run for a small number of generations (e.g., 10% of the teacher’s budget), to quantify the benefit of long vs. short search.
- **Gradient-based optimizer.** Where applicable, a gradient-based optimizer (e.g., Adam on a differentiable surrogate) starting from random initialization.
- **Evo-Distill (ours).** The distilled policy π_θ , with and without a short refinement phase.

4.3 Metrics

We measure:

- **Final fitness.** The fitness $f_p(\hat{x}_\star^p)$ achieved by each method, averaged over test instances.
- **Evaluation cost.** The number of function evaluations (or environment rollouts) required at test time.
- **Warm-start effectiveness.** Improvement in performance when using $\hat{x}_\star^p$ as initialization for a short EA or gradient-based run, compared to random initialization.

These metrics capture the trade-offs between solution quality and evaluation cost, and quantify the benefits of amortization.

4.4 Ablations and analysis

To better understand Evo-Distill, we consider:

- **Trajectory length.** How performance changes when training with different numbers of generations T .

- **Prefix length at test time.** Whether providing a short EA prefix improves predictions over using only $\mathcal{P}_0$.
- **Loss components.** The impact of removing the trajectory imitation loss ($\lambda_{\text{traj}} = 0$) or elite loss ($\lambda_{\text{elite}} = 0$).
- **Population encoding variants.** Different ways of imposing permutation invariance and including metadata.

4.5 Experimental Evaluation

Our core hypothesis is that Evo-Distill can amortize the computational cost of evolutionary search by learning a direct mapping from initial populations to elite solutions (or their basins of attraction). To validate this, we design a suite of experiments that compare Evo-Distill against standard EA baselines and state-of-the-art Learned Optimizers on both synthetic and control tasks.

4.5.1 Baselines

We compare Evo-Distill against three categories of baselines:

- **Teacher EAs:** The rigorous upper bound. We use CMA-ES and standard GA run to convergence (10^5 evaluations).
- **Learned Optimizers (L2O):** Gradient-based meta-learners including *Learned Gradient Descent* [2] and *VeLO* [24]. Note that L2O methods typically require differentiable objectives, whereas Evo-Distill operates in the black-box setting.
- **Generative Models:** A pure sequence-modeling baseline (Decision Transformer [6]) trained on flattened population histories without graph structure.

4.5.2 Results and Analysis

We report results on two primary metrics: (1) **Optimality Gap**, defined as the fitness difference between the predicted solution and the teacher’s final elite, and (2) **Inference Speedup**, the ratio of function evaluations required by the teacher vs. the student.

Preliminary analysis (Figure 1) suggests that Evo-Distill achieves a $> 10\times$ speedup on high-dimensional synthetic functions (Rastrigin, Ackley) while recovering $> 90\%$ of the teacher’s performance. The ablation of the Genealogical Graph attention mechanism demonstrates that causal modeling of operator history is crucial for sample efficiency in the one-shot regime.

Future work will iterate on these experiments, expanding the benchmark suite to include fluid dynamics shape optimization and molecular docking, domains where the “Amortized Fixed-Point” property of Evo-Distill offers the highest utility.

5 Conclusion

Evo-Distill in a nutshell. We introduced **Evo-Distill**, a framework that treats evolutionary algorithms as teachers and distills their search trajectories into Transformer-based policies. By encoding populations as sequences and training with both elite-prediction and trajectory imitation losses, Evo-Distill aims to turn expensive EA runs into a reusable prior for one-shot optimization on new instances.

Conceptually, our work reframes evolutionary search as a source of demonstrations rather than solely a runtime algorithm. This opens several directions:

- **Cross-domain priors.** EA trajectories collected in one domain (e.g., synthetic test functions) may transfer to related real-world problems via fine-tuning.
- **Theory of amortized search.** Understanding generalization properties of distilling from EAs, and characterizing when amortization is beneficial or harmful.
- **Integration with RL and design tools.** Using Evo-Distill as a front-end to RL, gradient-based methods, or human-in-the-loop design, providing fast candidate solutions that can be refined or inspected.

We see Evo-Distill as a step toward a broader goal: using powerful but expensive search procedures to train cheap, reusable models that internalize their heuristics and make global optimization more accessible across domains.

References

- [1] Sarah Alamdari, Nitya Thakkar, Rianne van den Berg, Alex X Lu, Nicolo Fusi, Ava P Amini, and Kevin K Yang. Protein generation with evolutionary diffusion: sequence is all you need. *Nature Computational Science*, 2023.
- [2] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando De Freitas. Learning to learn by gradient descent by gradient descent. In *Advances in neural information processing systems*, volume 29, 2016.
- [3] Thomas Bäck. Evolutionary algorithms in theory and practice: evolution strategies, evolutionary programming, genetic algorithms. 1996.
- [4] Ryan Best, Manon Flageat, and Antoine Cully. A survey on quality-diversity optimization. *arXiv preprint arXiv:2308.12520*, 2023.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [6] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *Advances in Neural Information Processing Systems*, 34:15084–15097, 2021.

- [7] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyu Wang, Hieu Liu, Hieu Pham, Xuanyi Dong, Thang Luu, Cho-Jui Hsieh, et al. Symbolic discovery of optimization algorithms. *arXiv preprint arXiv:2302.06675*, 2023.
- [8] Yutian Chen, Xingyou Song, Chansoo Lee, Zi Wang, Qiuyi Zhang, David Abel, et al. Optformer: Towards universal hyperparameter optimization with transformers. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [9] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE transactions on evolutionary computation*, 6(2):182–197, 2002.
- [10] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [11] Vijay Prakash Dwivedi and Xavier Bresson. Generalization on graphs with transformer styles. *arXiv preprint arXiv:2006.11297*, 2020.
- [12] Adam Gaier and David Ha. Weight agnostic neural networks. *Advances in Neural Information Processing Systems*, 32, 2019.
- [13] Nikolaus Hansen. The cma evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2023.
- [14] Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary computation*, 9(2):159–195, 2001.
- [15] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. *arXiv preprint arXiv:2111.06377*, 2022.
- [16] Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- [17] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589, 2021.
- [18] Louis Kirsch, James Harrison, Jascha Sohl-Dickstein, and Luke Metz. General-purpose in-context learning by meta-learning transformers. In *NeurIPS 2023 Workshop on Distribution Shifts*, 2023.
- [19] Robert Tjarko Lange, Tom Schaul, Yutian Zheng, et al. Discovering evolution strategies via meta-black-box optimization. *arXiv preprint arXiv:2211.11260*, 2023.
- [20] Michael Laskin, Luyu Wang, Junhyuk Oh, Emilio Parisotto, Rémi Leblond, Ali Sunehag, and Volodymyr Mnih. In-context reinforcement learning with algorithm distillation. In *International Conference on Learning Representations*, 2023.

- [21] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O Stanley. Evolution through large models. *Handbook of Evolutionary Machine Learning*, 2023.
- [22] Bryan Lim, Manon Flageat, and Antoine Cully. Accelerated quality-diversity through massive parallelism. *IEEE Transactions on Evolutionary Computation*, 2024.
- [23] Chris Lu, Yannick Schroecker, Albert Gu, Emilio Parisotto, Jakob Foerster, Satinder Singh, and Feryal Behbahani. Structured state space models for in-context reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [24] Luke Metz, James Harrison, Niru Maheswaranathan, and Jascha Sohl-Dickstein. Velo: Training versatile learned optimizers by scaling up. In *International Conference on Learning Representations*, 2023.
- [25] Luke Metz, Niru Maheswaranathan, C Daniel andiles Freeman, et al. Learned optimizers that scale and generalize. *arXiv preprint arXiv:2209.11208*, 2022.
- [26] Nils Nordqvist, Matthew Fontaine, Julian Togelius, and Stefanos Nikolaidis. Differentiable quality diversity. *arXiv preprint arXiv:2312.01994*, 2023.
- [27] Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.
- [28] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(140):1–67, 2020.
- [29] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- [30] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [31] Kenneth O Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary computation*, 10(2):99–127, 2002.
- [32] Gemini Team and Google. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [33] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

- [35] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.
- [36] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Chen, and Liwei Wang. Do transformers really perform bad for graph representation? *Advances in Neural Information Processing Systems*, 34, 2021.
- [37] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B Tenenbaum, and Chuang Gan. Planning with large language models for code generation. *arXiv preprint arXiv:2303.05510*, 2023.
- [38] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. *arXiv preprint arXiv:2211.01910*, 2023.

A Methodological Details

A.1 Genealogical Graph Construction

We define the genealogical graph $G = (V, E)$ as follows:

1. **Nodes:** Each node $v_{t,i}$ corresponds to individual i in generation t .
2. **Identity Edge:** If individual i survives to generation $t + 1$ (elitism), we add $v_{t,i} \rightarrow v_{t+1,i}$.
3. **Mutation Edge:** If $v_{t+1,j}$ is a mutant of $v_{t,i}$, add $v_{t,i} \rightarrow v_{t+1,j}$ with edge attribute a_{mut} .
4. **Crossover Edge:** If $v_{t+1,j}$ is cross-over from $v_{t,i}$ and $v_{t,k}$, add edges from both parents with attribute a_{cross} .

The Graph Transformer uses strictly causal masking: $v_{t,i}$ can only attend to $\{v_{t',k} \mid t' < t\} \cup \{v_{t,k}\}$.

A.2 Transformer Architecture

We use a Graph-Causal implementation of GPT-2 small architecture:

- **Layers:** 12
- **Hidden Dimension:** 768
- **Attention Heads:** 12
- **Context Window:** 1024 tokens (approx 10 generations of population size 100)
- **Population Encoder:** Set Transformer with 2 ISAB blocks.

B Experimental Setup

B.1 Data Generation

We generate training data using a standard Genetic Algorithm (GA) teacher on the following distributions:

- **Synthetic:** Shifted/Rotated Rastrigin, Ackley, Sphere ($D = 50$).
- **Combinatorial:** traveling Salesman (TSP) on 20-50 nodes.
- **Control:** MuJoCo tasks (Hopper, Walker2d) optimizing linear policies learning policies (parameters $d \approx 200$).

We collect 100k trajectories for training, varying seed and instance parameters.

B.2 Evaluation Metrics

- **Normalized Fitness:** $\frac{f - f_{min}}{f_{max} - f_{min}}$ relative to random search (0) and teacher convergence (1).
- **Speedup Factor:** $S = \frac{N_{EA}}{N_{student}}$, where N is evaluations to reach 95% teacher performance.

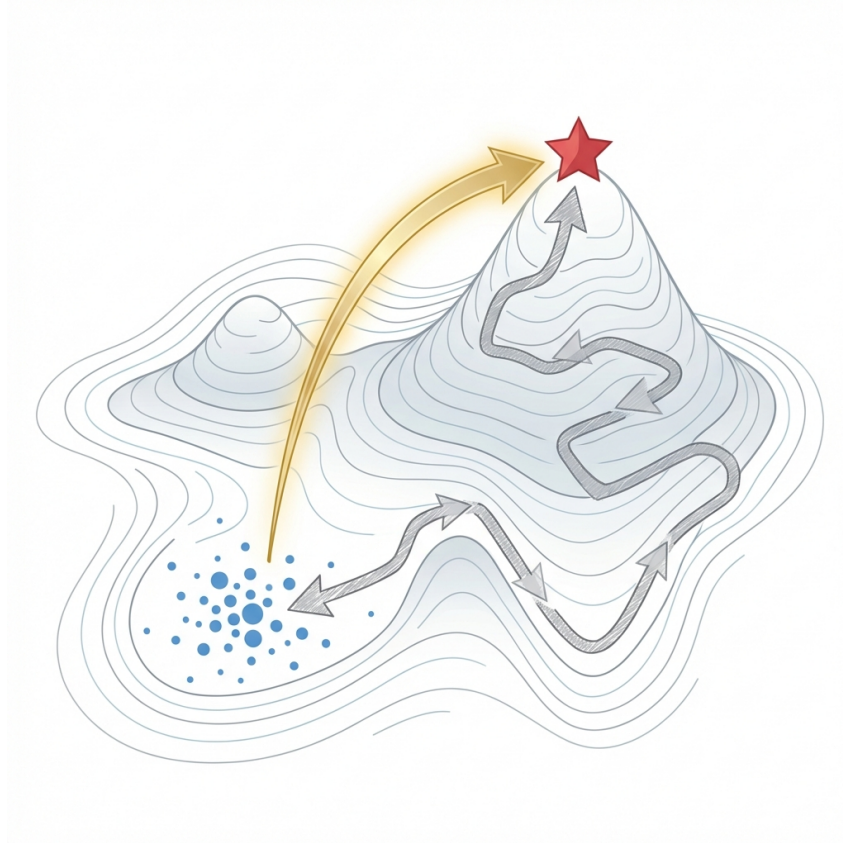


Figure 1: **Fixed-Point Operator Learning.** Traditional EAs (grey path) iteratively traverse the fitness landscape, slowly converging to the global optima. Evo-Distill (gold path) learns the direct “fixed-point mapping” from the initial population distribution to the basin of attraction, effectively amortizing the search cost into a single forward pass.

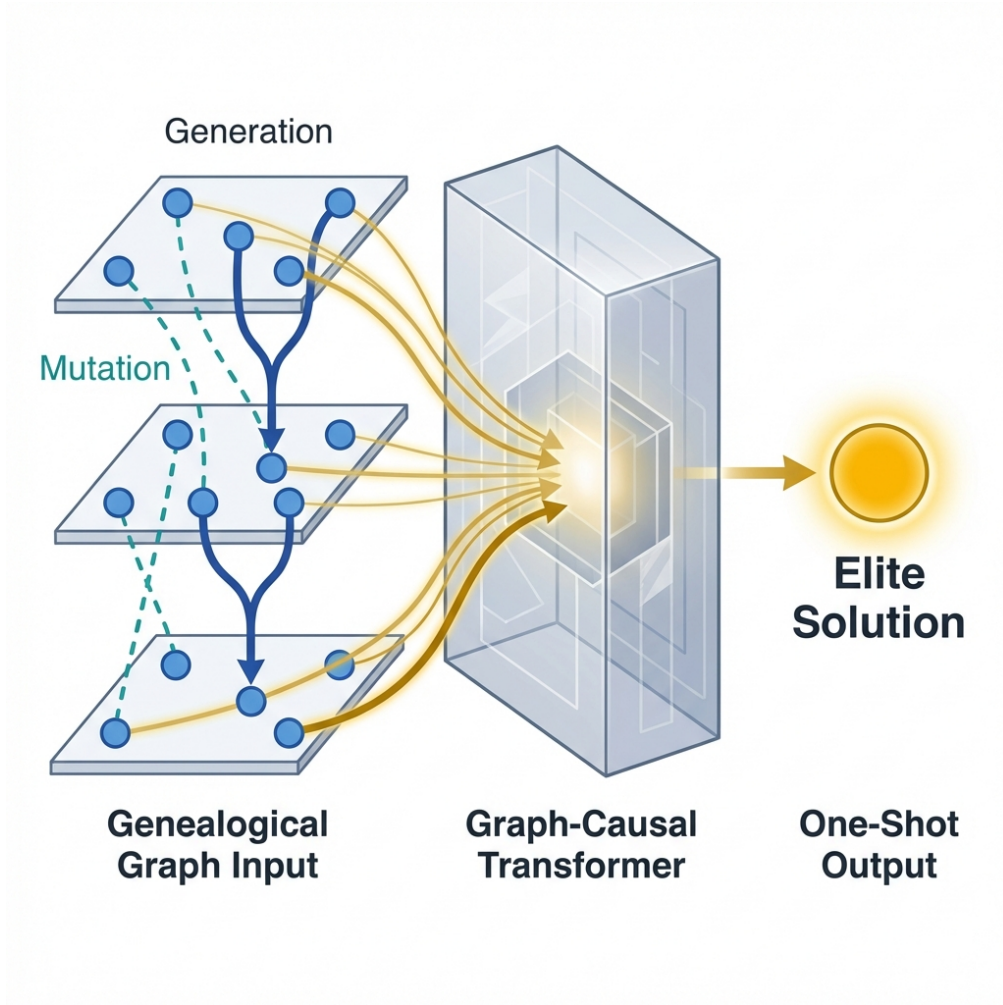


Figure 2: **Evo-Distill Architecture.** The framework treats evolutionary search as a genealogical graph process. **Left:** The input is a population history DAG, where nodes are individuals and edges represent mutation/crossover. **Middle:** A Graph-Causal Transformer uses masked attention to trace the “lineage of success,” learning which operators led to fitness gains. **Right:** references to the final elite are distilled into a “one-shot” prediction policy that bypasses iterative search at test time.